

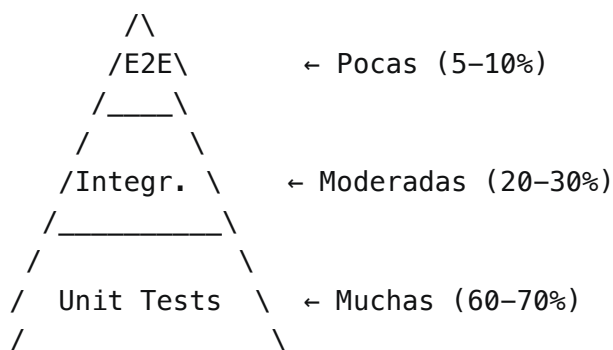
Lección 1: Testing Setup

Setup + Mapa de Pruebas

Agenda

- La Pirámide de Testing
 - Tipos de Pruebas
 - Herramientas: Vitest, Testing Library, Playwright
 - Estrategias de Coverage
 - Mejores Prácticas
-

La Pirámide de Testing



Principio: Más tests unitarios, menos E2E

Unit Tests: Características

- **Rápidas:** < 10ms por test

- **Aisladas:** Sin dependencias externas
- **Determinísticas:** Mismo input = mismo output
- **Fáciles de mantener:** Cambios localizados

Cuándo usar: Funciones puras, lógica de negocio, cálculos, validaciones

Integration Tests: Características

- **Moderadamente rápidas:** 100-500ms por test
- **Conectadas:** Múltiples componentes juntos
- **Funcionales:** Validan flujos de usuario
- **UI-centric:** Usan DOM real

Cuándo usar: Flujos de usuario, interacciones entre componentes, estados compartidos

E2E Tests: Características

- **Lentas:** 1-10 segundos por test
- **Reales:** Browser + Network + Backend
- **Costosas:** Infraestructura compleja
- **Críticas:** Validan flujos completos

Cuándo usar: Flujos críticos de negocio, validación cross-browser, regresiones

Vitest: El Runner Moderno

¿Por qué Vitest?

- ⚡ Velocidad: 10x más rápido que Jest
 - 🔥 Hot Reload: Tests se re-ejecutan automáticamente
 - 📊 Coverage nativo: Sin configuración adicional
 - 🎯 ESM First: Soporte nativo para módulos ES
-

Testing Library: User-Centric

Philosophy:

“The more your tests resemble the way your software is used, the more confidence they can give you.”

Query Hierarchy:

1. `getByRole` - Más semántico
 2. `getByLabelText` - Para formularios
 3. `getByText` - Para contenido visible
 4. `getByTestId` - Último recurso
-

Queries en Testing Library (1/2)

¿Qué son las queries? Funciones que buscan elementos en el DOM como lo haría un usuario real.

Tipos de queries:

- `getBy*` : Encuentra elemento, falla si no existe (assertions)
 - `queryBy*` : Encuentra elemento, retorna `null` si no existe (verificar ausencia)
 - `findBy*` : Encuentra elemento async, espera hasta que aparezca (loading)
-

Queries en Testing Library (2/2)

Ejemplo:

```
// Usuario ve botón por su texto
screen.getByRole('button', { name: /add to cart/i })

// Usuario ve input por su label
screen.getByLabelText('Email')

// Verificar elemento NO existe
expect(screen.queryByText('Error')).toBeNull()
```

Playwright: E2E de Nueva Generación

Ventajas:

- 🚀 3x más rápido que Selenium
 - 🤖 Multi-browser automático
 - 📱 Emulación de dispositivos móviles
 - 🎥 Auto-screenshots y videos
 - 🔍 Auto-wait inteligente
-

¿Qué es Coverage?

Coverage mide qué porcentaje del código ejecutan tus tests

4 Métricas de Coverage:

1. **Functions:** % de funciones ejecutadas

```
function calculateTotal() {} // ¿Tests llaman esta función?
```

2. **Lines:** % de líneas de código ejecutadas

```
const subtotal = calculateSubtotal(items) // ¿Tests ejecutan esta línea
```

Coverage Metrics (2/2)

3. **Branches:** % de caminos en condicionales

```
if (quantity >= 5) {  
  // ¿Tests prueban AMBOS caminos?  
  return discount // ← Camino 1  
} else {  
  return 0 // ← Camino 2  
}
```

4. Statements: % de declaraciones ejecutadas

```
const x = 10 // Statement 1
return x * 2 // Statement 2
```

Estrategia de Coverage

```
coverage: {
  thresholds: {
    global: {
      functions: 100,    // Todas las funciones
      lines: 80,        // 80% líneas
      branches: 80,     // 80% branches
      statements: 80    // 80% statements
    }
  }
}
```

100% funciones, 80% líneas es realista

Métricas de Éxito

Cuantitativas (medibles):

- Coverage: Functions 100%, Lines 80%+
- Velocidad: Unit < 10ms, Integration < 500ms

Cualitativas (perceptibles):

- Confianza para refactorizar
 - Tests como documentación viviente
 - Retroalimentación rápida (< 30s)
-

Ejercicio 1: Unit Test con AAA Pattern

Prompt:

Actúa como un desarrollador senior que escribe unit tests.

CONTEXT0: Los unit tests deben ser rápidos (<10ms), aislados, determinísticos y fáciles de mantener. Son ideales para funciones puras y lógica de negocio.

TAREA: Crea una función `calculateSubtotal` y su `test` unitario.

ESPECIFICACIONES:

- Función: recibe array de `CartItem` `{price: number, quantity: number}`
- Lógica: suma (`price × quantity`) de todos los items
- Implementación: usa `Array.reduce()`

TEST REQUIREMENTS:

- Framework: Vitest (`import { describe, it, expect } from 'vitest'`)
- Estructura: patrón AAA (Arrange-Act-Assert) con comentarios
- Test data: `[{price: 10, quantity: 2}, {price: 5.50, quantity: 1}]`
- Resultado esperado: 25.50

ARCHIVOS:

- `src/shared/utils/calculateSubtotal.ts` (función)
- `src/shared/utils/calculateSubtotal.test.ts` (`test`)

Aprende: AAA pattern estructura tests claros y mantenibles

Ejercicio 2: Integration Test

Prompt:

Actúa como un desarrollador que practica user-centric testing.

CONTEXT0: Los integration tests validan flujos de usuario con múltiples componentes conectados. Deben usar DOM real y buscar elementos como lo haría un usuario (priorizando `getByRole` sobre `getById`).

TAREA: Crea componente `SimpleCounter` y su integration `test`.

COMPONENTE REQUIREMENTS:

- Estado inicial: `count = 0`
- UI: muestra `"Count: {count}"` con `data-testid="count-display"`
- Acción: botón `"Increment"` incrementa `count` en 1

- Framework: React con useState

TEST REQUIREMENTS:

- Framework: Vitest + Testing Library
- Imports: render, screen, fireEvent desde @testing-library/react
- Query prioritaria: getByRole('button', { name: /increment/i })
- Interacción: fireEvent.click() para simular click del usuario
- Assertion: expect().toHaveTextContent() para verificar texto
- Estructura: sigue patrón AAA con comentarios

ARCHIVOS:

- src/features/counter/SimpleCounter.tsx (componente)
- src/features/counter/SimpleCounter.test.tsx (test)

VALIDACIÓN: ejecuta pnpm test para verificar

Aprende: getByRole busca elementos como lo haría

un usuario real

Puntos Clave

1. **Testing Pyramid:** Más unit, menos E2E
2. **Centrado en el Usuario:** Testa comportamiento, no implementación
3. **Coverage:** 100% functions, 80% lines
4. **Retroalimentación Rápida:** Tests rápidos = desarrollo rápido
5. **Herramientas modernas:** Vitest + Testing Library + Playwright